AWS Identity and Access Management

EC2 Access Control Using IAM

IAM Task 1

Platform:	Amazon Web Services (AWS)
Service:	Identity and Access Management (IAM)
Resource:	Amazon EC2
IAM User:	USER1
IAM Group:	EC2-K
Region:	ap-south-1 (Mumbai)

Practical Implementation Report

2026

1 Introduction

AWS IAM is used to decide who can use AWS resources and what they can do.

In this experiment, we created a user called `USER1` and gave the user limited access to EC2.

The user should be able to:

- View EC2 resources.
- Start the Target instance.
- Stop the Target instance.
- Not control the Secondary instance.
- Not terminate an EC2 instance.

To do this, we used an IAM user, an IAM group, and three policies.

- `AmazonEC2ReadOnlyAccess`
- `EC2-K-Control`
- `EC2-k-Terminate`

After creating the setup, we tested the permissions using the EC2 console.

2 Problem Statement

The task was to create a limited EC2 user.

The user should have enough access to manage one particular EC2 instance, but should not be able to control other instances or delete an instance.

The main requirements were:

1. Give the user read-only EC2 access.
2. Allow the user to start and stop one specific EC2 instance.
3. Do not allow the user to start or stop other instances.
4. Do not allow the user to terminate an instance.
5. Give the permissions through an IAM group.

3 Resources Used

The following AWS resources were used.

Table 1: Resources used

Resource	Value
IAM User	USER1
IAM Group	EC2-K
AWS Region	ap-south-1
Target Instance	i-0b3f8daef4437021c
Secondary Instance	i-0186ea89cc5788b46
Instance Type	t3.micro
Read-only Policy	AmazonEC2ReadOnlyAccess
Control Policy	EC2-K-Control
Safety Policy	EC2-k-Terminate

4 EC2 Setup

Before creating the IAM permissions, two EC2 instances were available.

One instance was named `Target` and the other was named `Secondary`.

The Target instance was used to check whether USER1 could start and stop the correct instance.

The Secondary instance was used to check whether USER1 could incorrectly control another instance.

4.1 EC2 Instances

The EC2 console showed both instances.

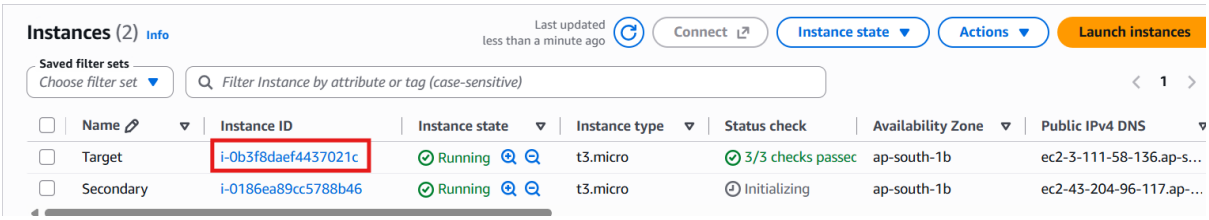


Figure 1: Target and Secondary EC2 instances

The Target instance ID was:

`i-0b3f8daef4437021c`

The details of the Target instance are shown below.

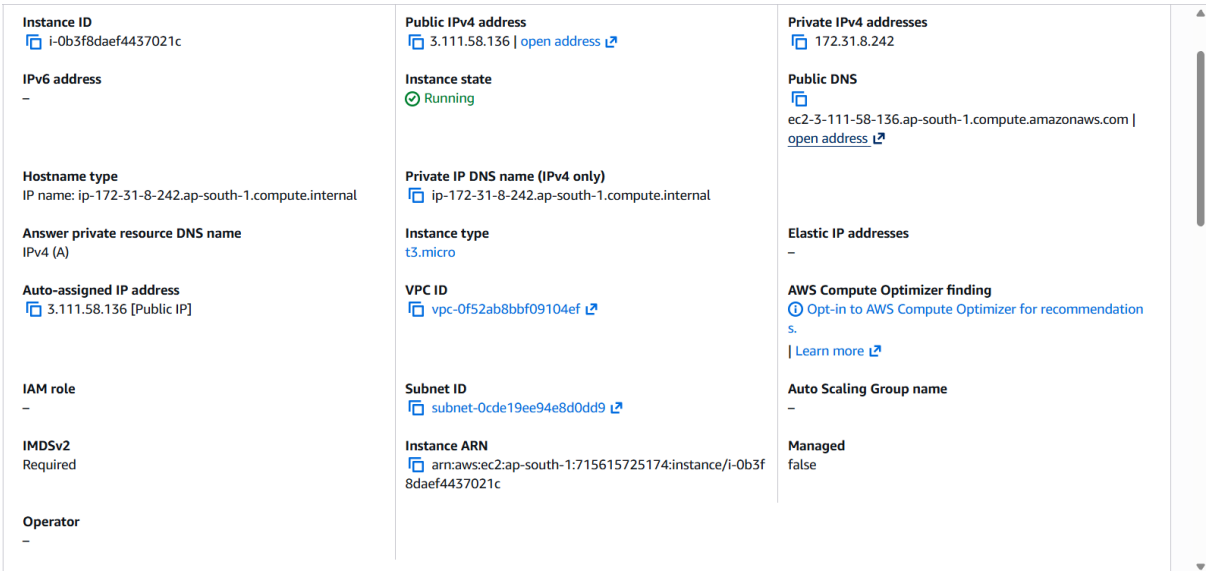


Figure 2: Details of the Target EC2 instance

5 Creating the IAM User

A new IAM user named `USER1` was created.

No permissions were directly added to the user.

Instead, the permissions were later given through an IAM group.

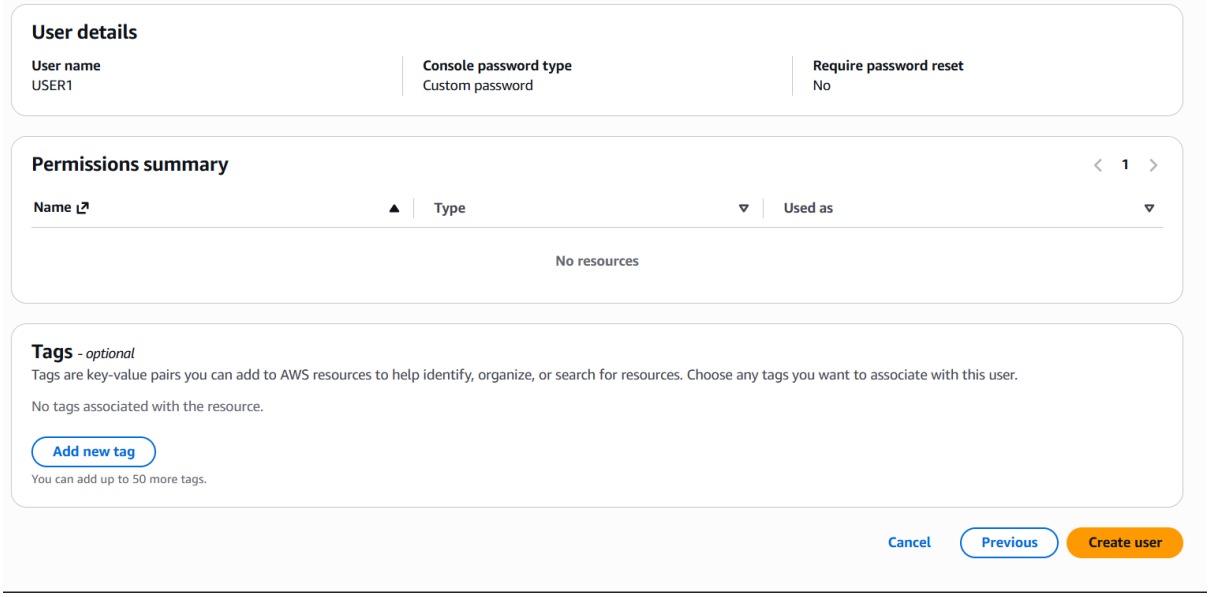


Figure 3: Creating USER1

The IAM users page shows that USER1 was created successfully.

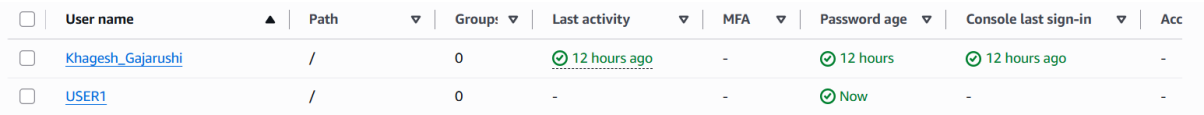


Figure 4: USER1 in the IAM users list

6 Creating the IAM Group

An IAM group named:

EC2-K

was created.

The reason for using a group is simple: instead of giving permissions separately to every user, we can give the permissions to the group.

Any user added to the group will get the same permissions.

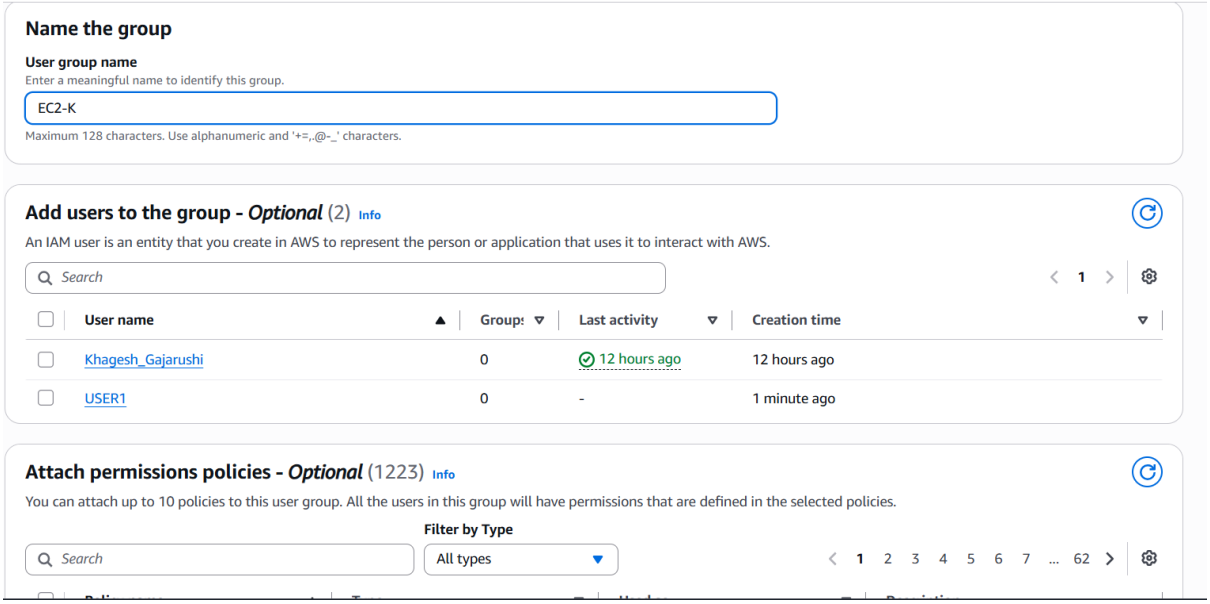


Figure 5: Creating the EC2-K group

USER1 was added to the group.

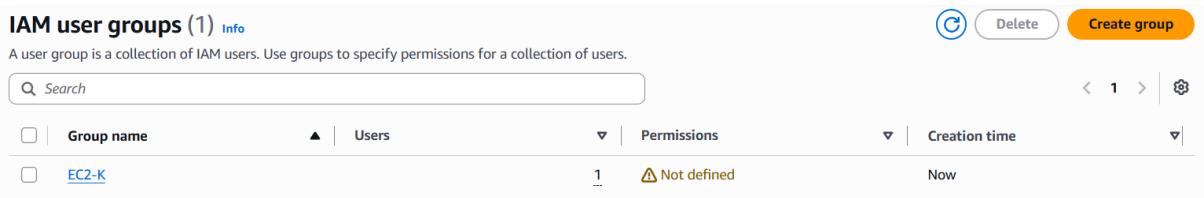


Figure 6: EC2-K group with USER1

7 Adding Read-Only EC2 Access

The first policy added to the group was:

`AmazonEC2ReadOnlyAccess`

This allows USER1 to view EC2 information without giving permission to change the instances.

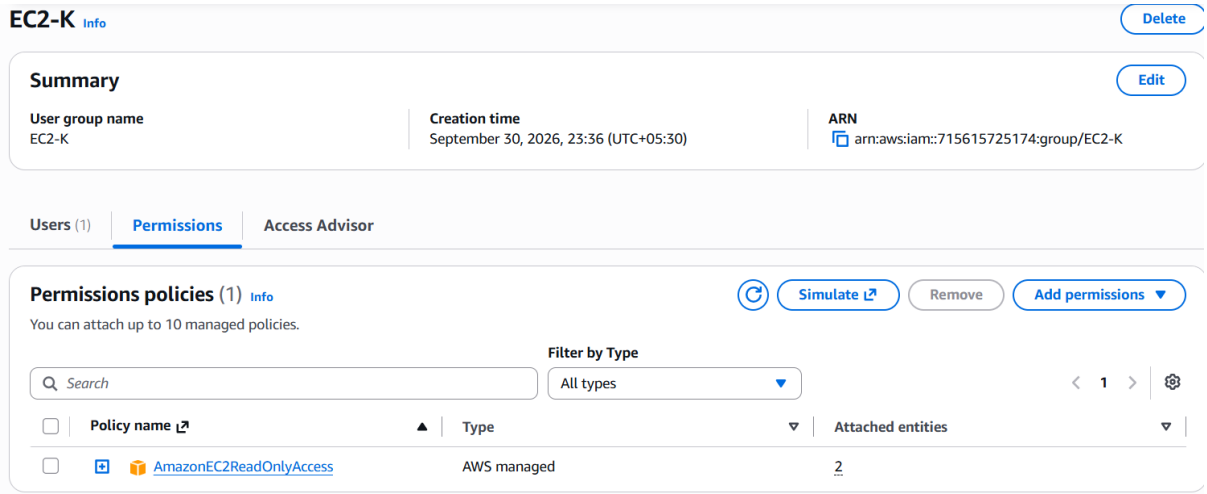


Figure 7: Adding AmazonEC2ReadOnlyAccess

So USER1 can see the EC2 resources, but this policy alone does not allow USER1 to start or stop instances.

8 Creating the Start and Stop Policy

The next requirement was to allow USER1 to start and stop only the Target instance.

We created a custom policy called:

`EC2-K-Control`

Only these two actions were selected:

```
ec2:StartInstances
ec2:StopInstances
```

The policy was also limited to the Target instance.

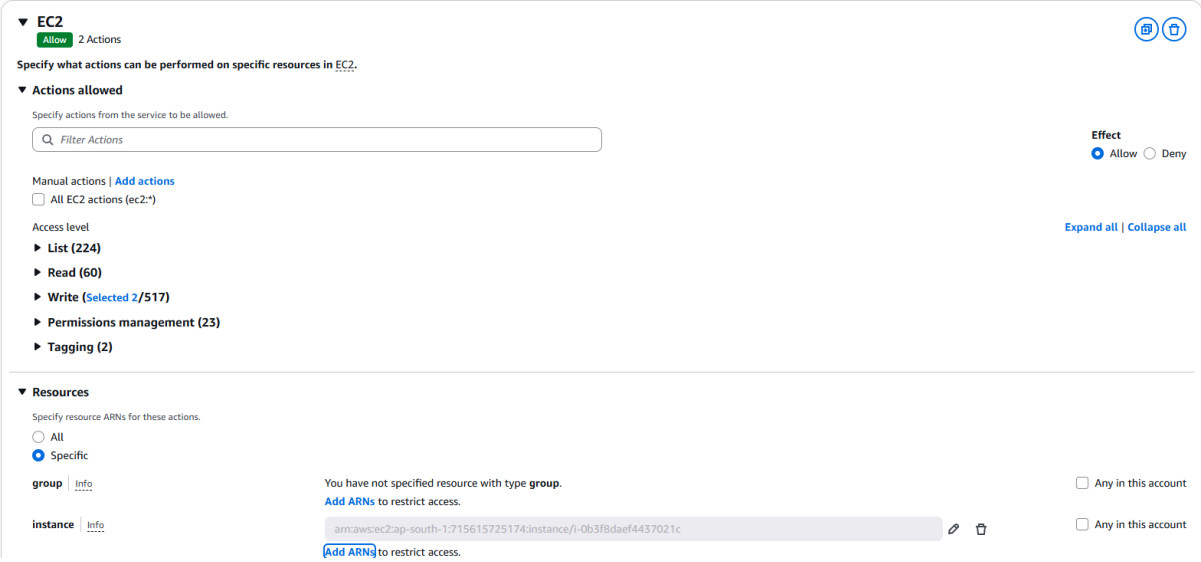


Figure 8: Selecting Start and Stop actions

The policy was given the name `EC2-K-Control`.

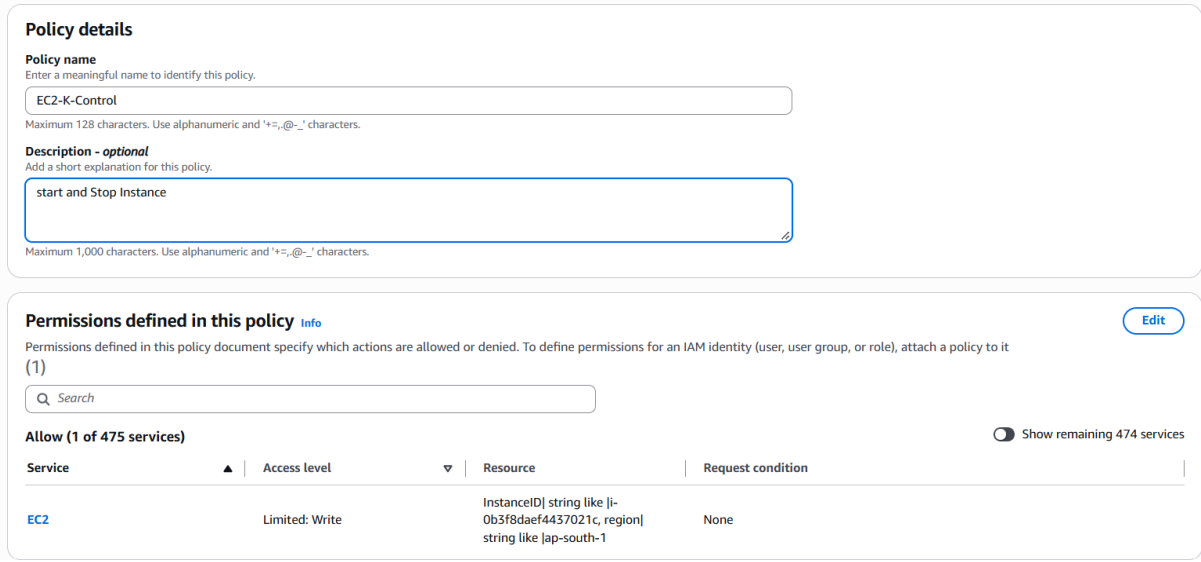


Figure 9: EC2-K-Control policy details

The policy was created successfully.

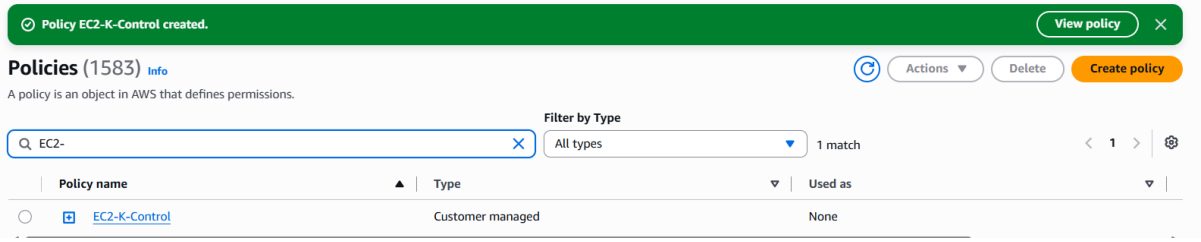


Figure 10: EC2-K-Control policy created

9 Blocking Instance Termination

Starting and stopping an instance was allowed, but we did not want USER1 to terminate an instance.

So, another policy was created.

The policy was called:

`EC2-k-Terminate`

The policy was set to:

```
Effect: Deny
Action: ec2:TerminateInstances
Resource: All resources
```

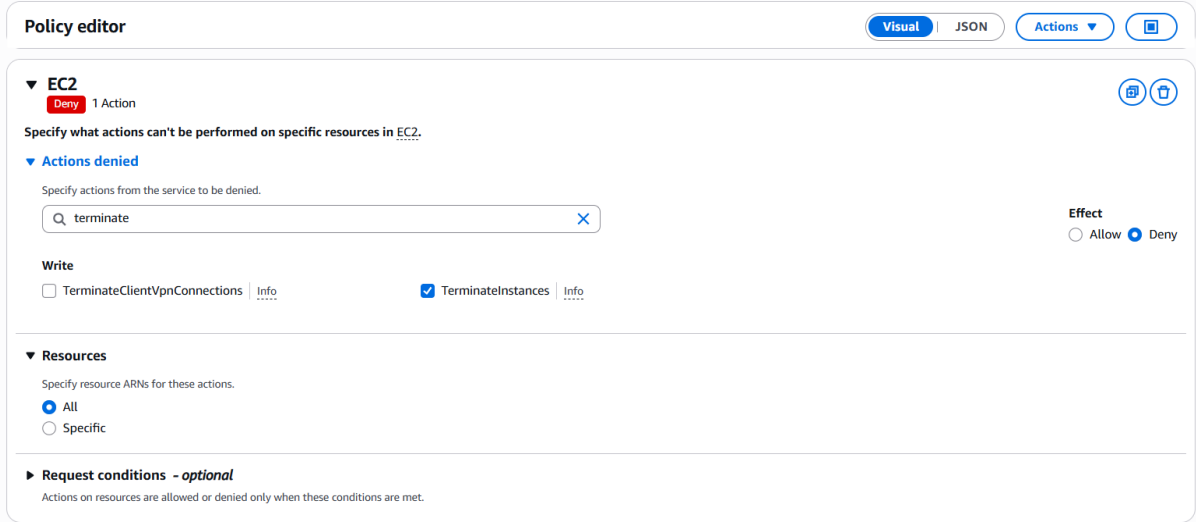


Figure 11: Blocking the TerminateInstances action

The policy name and description were entered.

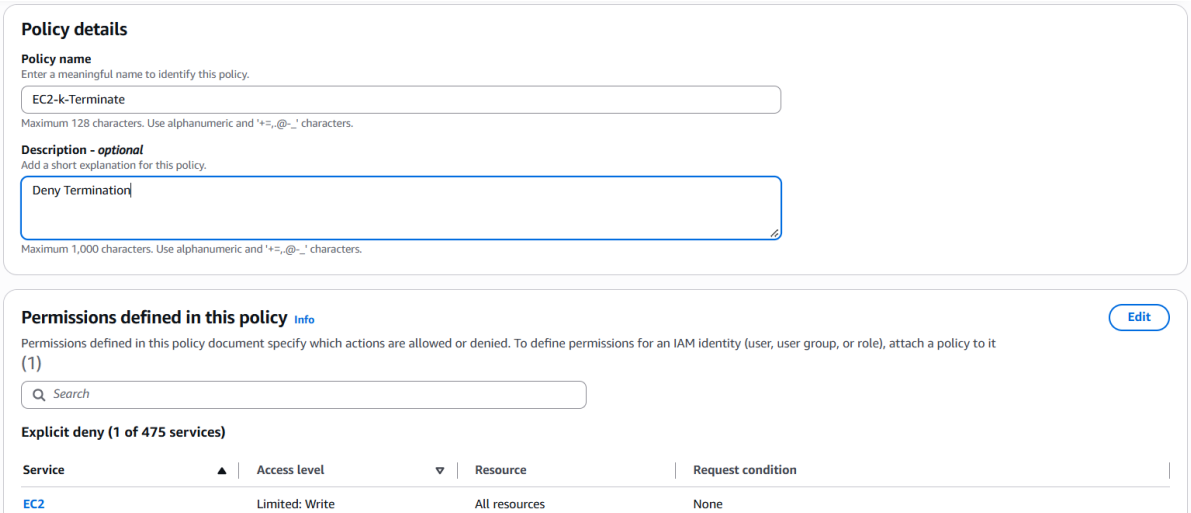


Figure 12: Termination policy details

The policy was created successfully.

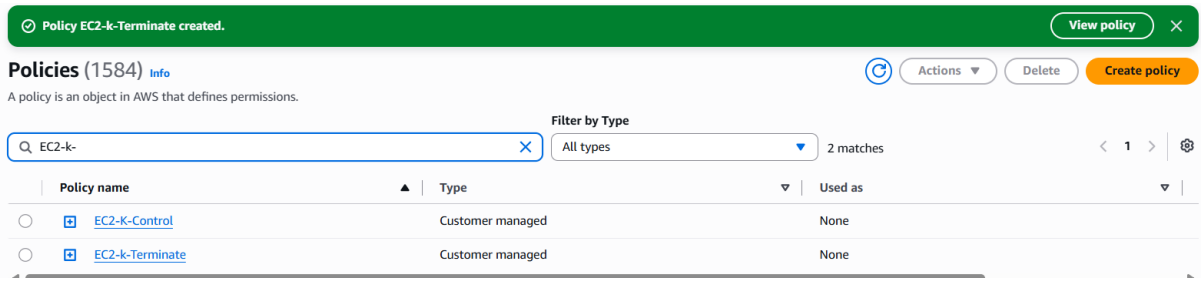


Figure 13: EC2-k-Terminate policy created

10 Final Group Configuration

After creating all the policies, they were attached to the `EC2-K` group.

The group now had three policies:

1. `AmazonEC2ReadOnlyAccess`
2. `EC2-K-Control`
3. `EC2-k-Terminate`

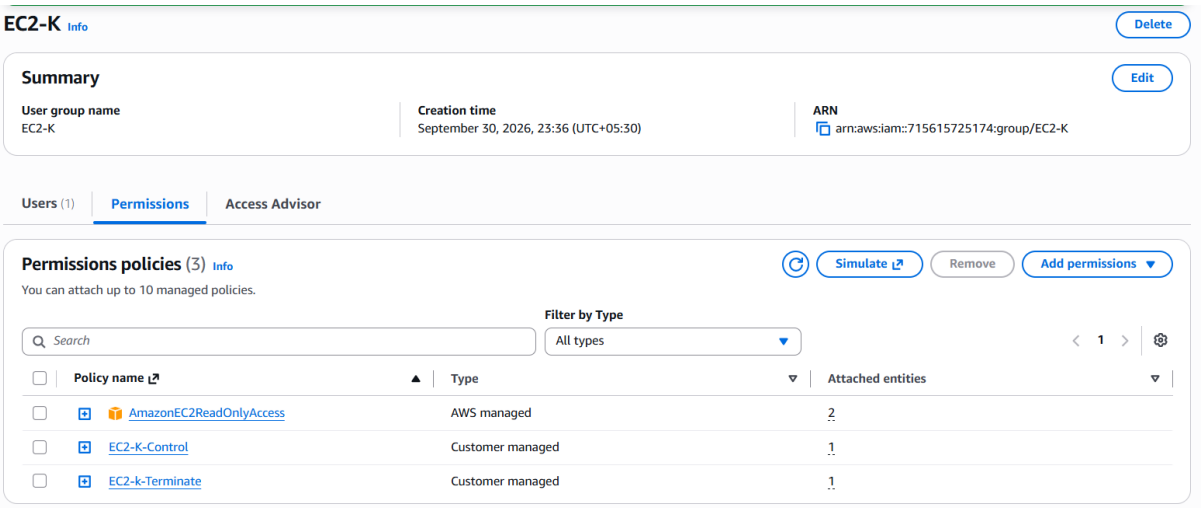


Figure 14: Final EC2-K group permissions

The final setup was now ready for testing.

11 Question 1: How did we give access only to the required instance?

Question

How can we allow a user to start and stop one EC2 instance without allowing the user to control other instances?

Our Solution

We created the policy:

`EC2-K-Control`

This policy only allows:

```
ec2:StartInstances
ec2:StopInstances
```

We then selected the Target instance as the resource.

So USER1 can start and stop the Target instance.

USER1 does not get the same permission for every EC2 instance.

In simple words:

USER1 can control Target, but not other instances.

Testing the Target Instance

Initially, both EC2 instances were running.

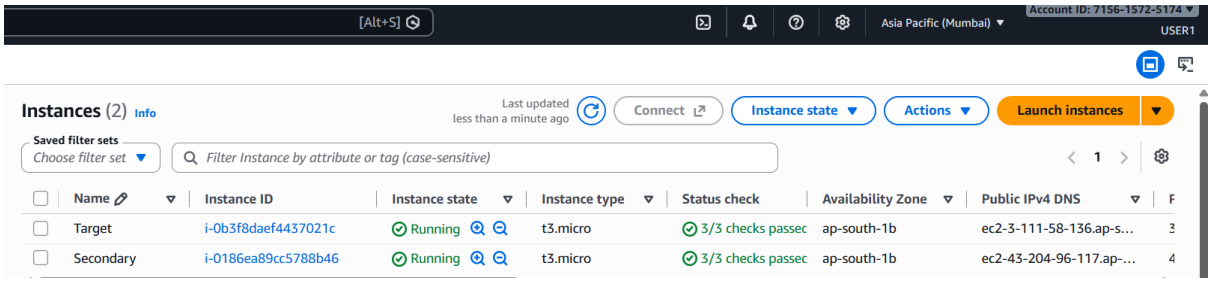


Figure 15: Both EC2 instances running

USER1 then tried to stop the Target instance.

The operation was successful.

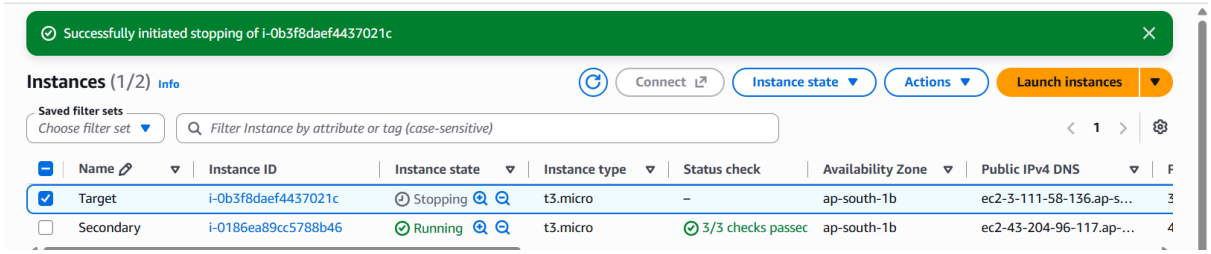


Figure 16: Target instance being stopped

This shows that USER1 has the required permission for the Target instance.

Testing the Secondary Instance

Next, USER1 tried to stop the Secondary instance.

The operation failed.

AWS showed an error saying that USER1 was not allowed to perform the operation.

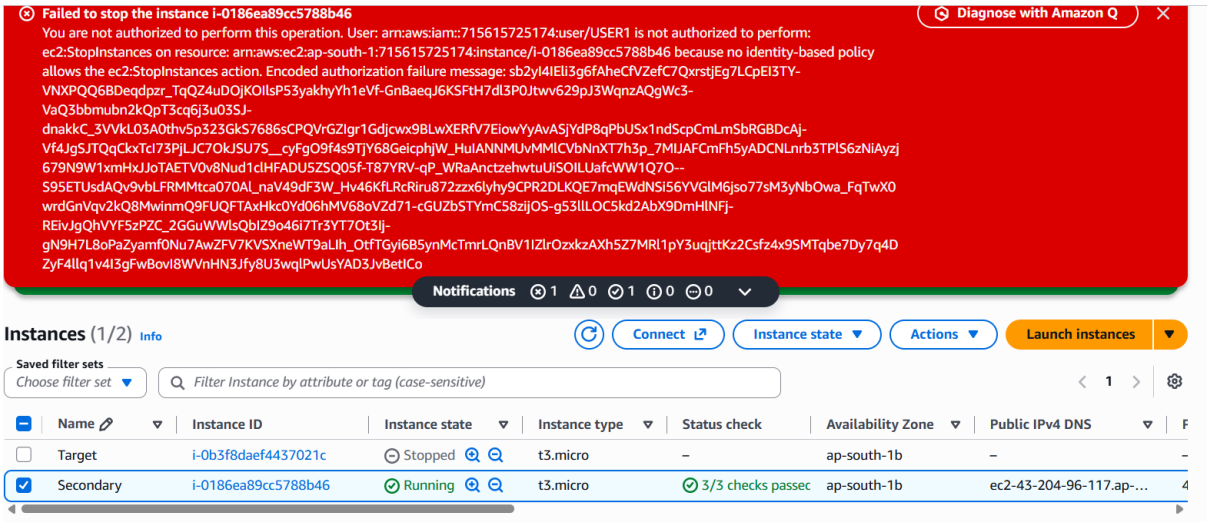


Figure 17: USER1 was not allowed to stop the Secondary instance

This proves that the permission is limited to the Target instance.

Result

Target: Allowed **Secondary: Denied**

12 Question 2: How did we stop the user from terminating an instance?

Question

How can we make sure that USER1 cannot terminate an EC2 instance?

Our Solution

We created another policy:

`EC2-k-Terminate`

This policy contains an explicit Deny for:

```
ec2:TerminateInstances
```

The important part is the word **Deny**.

AWS blocks the action when an explicit Deny applies.

So even if another policy later gives permission to terminate an instance, this Deny will still block it.

Testing Termination

USER1 tried to terminate the Target instance.

AWS rejected the request.

The console showed an authorization error.

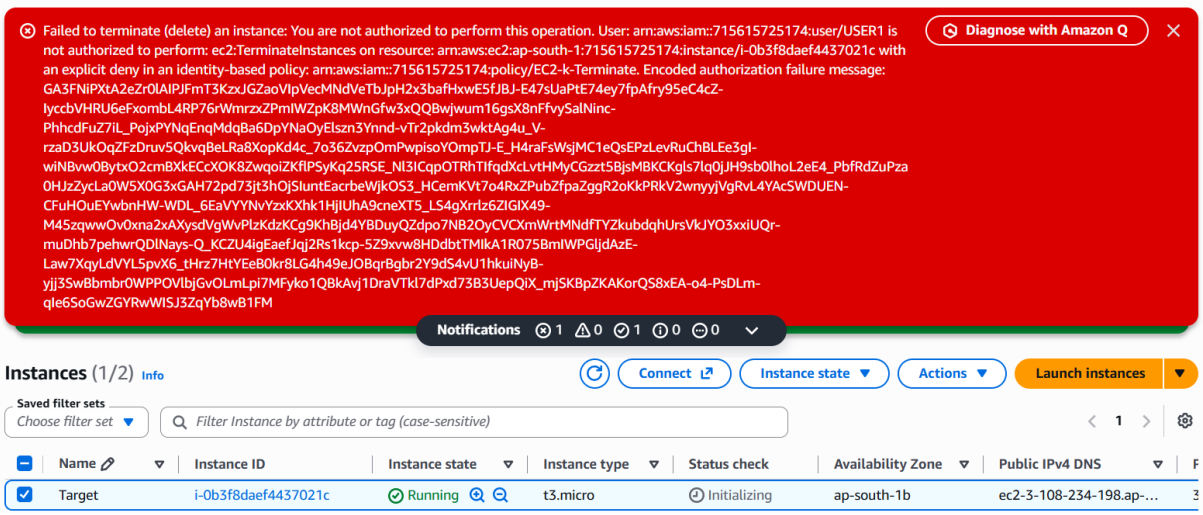


Figure 18: Termination attempt blocked by AWS

Result

USER1 can start and stop the Target instance, but cannot terminate it.

13 Question 3: Why did we use an IAM group?

Question

How can we manage the same permissions for multiple users without adding every policy to every user separately?

Our Solution

We created the IAM group:

`EC2-K`

USER1 was added to this group.

The policies were attached to the group instead of directly attaching them to USER1.

The setup is:

USER1

↓

EC2-K Group

↓

AmazonEC2ReadOnlyAccess
EC2-K-Control
EC2-k-Terminate

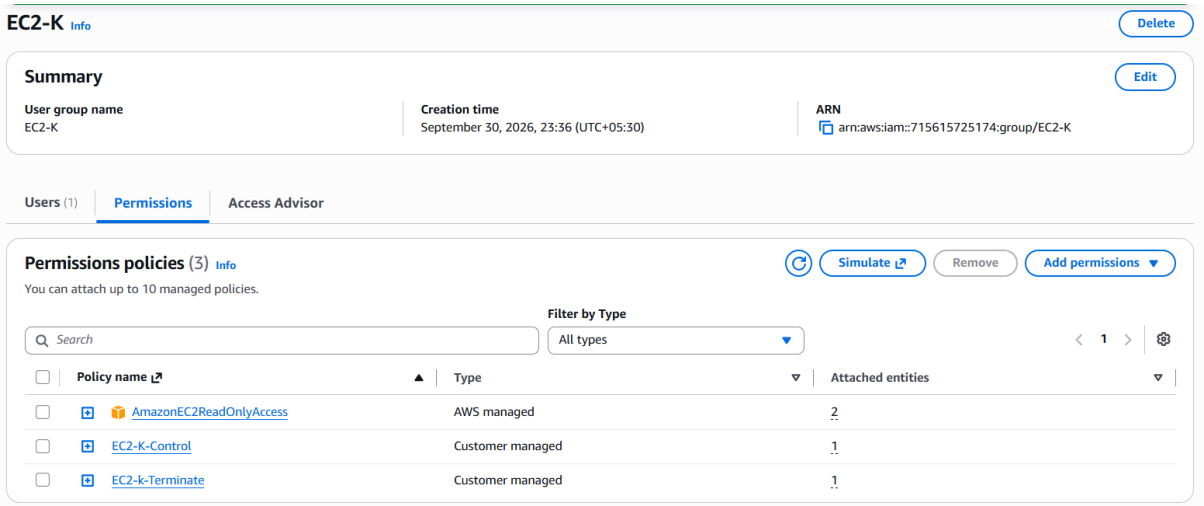


Figure 19: Policies attached to the EC2-K group

If another user needs the same permissions, we can simply add that user to the `EC2-K` group.

This makes the setup easier to manage.

14 Testing the Start Operation

After stopping the Target instance, USER1 tried to start it again.

The start operation was successfully initiated.

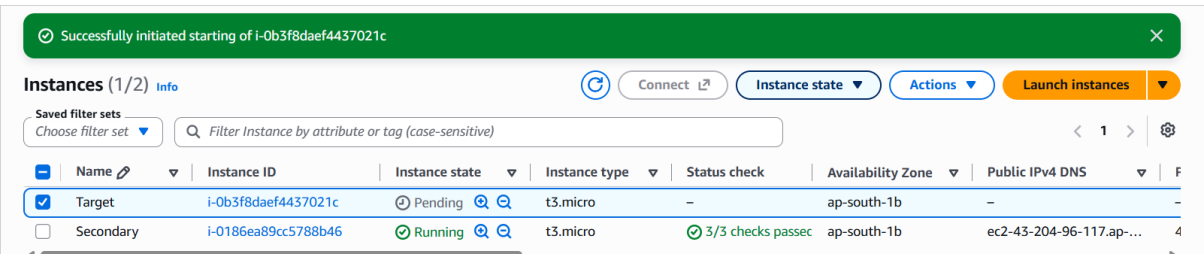


Figure 20: Target instance successfully started

This confirms that USER1 has the required Start permission for the Target instance.

15 Final Test Results

The tests gave the following results.

Table 2: Final test results

Test	Result	Why
View EC2 resources	Allowed	Read-only policy was attached
Start Target	Allowed	Start permission was given for Target
Stop Target	Allowed	Stop permission was given for Target
Stop Secondary	Denied	Permission was limited to Target
Terminate Target	Denied	Termination was explicitly denied

16 How Our Implementation Solved the Three Questions

The three requirements were solved using three simple parts.

Question 1 – Control only the required instance

We created `EC2-K-Control`.

It allows:

```
StartInstances
StopInstances
```

Only for the Target instance.

Because of this, USER1 could control the Target but not the Secondary instance.

Question 2 – Stop dangerous actions

We created `EC2-k-Terminate`.

It contains:

```
Deny: TerminateInstances
```

Because of this, USER1 could not terminate an EC2 instance.

Question 3 – Manage permissions using a group

We created the `EC2-K` group.

USER1 was added to the group and the three policies were attached to it.

This means another user can receive the same permissions simply by joining the group.

17 Final Setup

The final setup can be shown simply as:

USER1

↓

IAM Group: EC2-K

↓

`AmazonEC2ReadOnlyAccess`
Read EC2 information

`EC2-K-Control`
Start and Stop Target

`EC2-k-Terminate`
Block Termination

18 What We Learned

This experiment showed a few important IAM concepts.

18.1 Give Only the Access Needed

USER1 only needed to view EC2 and start or stop the Target instance.

So we gave only those permissions.

USER1 was not given permission to control every EC2 instance.

18.2 Limit the Permission to One Instance

It is not enough to say that a user can start or stop an instance.

We also need to decide **which instance** the user can control.

In our case, the permission was limited to the Target instance.

That is why the Target could be stopped but the Secondary instance could not.

18.3 Block Important Actions

We did not want USER1 to terminate an instance.

So we created a separate policy that explicitly blocks:

```
ec2:TerminateInstances
```

When USER1 tried it, AWS blocked the request.

18.4 Use Groups

Instead of adding the policies directly to USER1, we added them to the `EC2-K` group.

This makes it easier to give the same permissions to another user later.

19 Conclusion

In this experiment, we created a restricted IAM user for EC2.

We created:

- IAM user: `USER1`
- IAM group: `EC2-K`
- Read-only policy: `AmazonEC2ReadOnlyAccess`
- Start/Stop policy: `EC2-K-Control`
- Termination block policy: `EC2-k-Terminate`

The final tests showed that:

1. USER1 could view EC2 resources.
2. USER1 could start the Target instance.
3. USER1 could stop the Target instance.
4. USER1 could not stop the Secondary instance.
5. USER1 could not terminate the Target instance.

So, the required access was given to USER1 while unwanted actions were blocked.

The experiment also showed how IAM groups can be used to manage permissions more easily.

End of Experiment